

Ejercicio 1 — Cambiar la imagen de predicción en YOLO

Enlaces del notebook en colab

https://colab.research.google.com/drive/19W2VA9Hh1P8B_ayG9NBJ96PFStUtyHoz#scrollTo=b0rFzr_Oz1cl

Salidas

Zidane

The screenshot shows a Google Colab notebook interface. The left pane contains code cells for installing ultralytics, running inference on a specific image, and a detection summary. The right pane displays the output image with bounding boxes and confidence scores for detected objects.

```
[1] # Run inference on an image with YOLOv8n
ultralytics predict model=yolov8n.pt source="https://ultralytics.com/images/zidane.jpg"
```

Downloaded <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to 'yolov8n.pt': 100%
Ultralytics 8.4.150 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

Downloaded <https://ultralytics.com/images/zidane.jpg> to 'zidane.jpg': 100%
Image 1/1 /content/zidane.jpg: 384x640 2 persons, 1 tie, 6.3ms
Speed: 189.5ms preprocess, 6.3ms inference, 39.2ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /content/runs/detect/predict
Learn more at <https://docs.ultralytics.com/modes/predict>

1. Detection

YOLOv8 detection models have no suffix and are the default YOLOv8 models, i.e. yolov8n.pt and are pretrained on COCO. See [Detection Docs](#) for full details.

```
[4] # Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO
```

Output image (zidane.jpg) showing bounding boxes and confidence scores:

- person 0.84
- person 0.82

Bus

The screenshot shows a Google Colab notebook interface. The left pane contains code cells for loading a pre-trained YOLOv8n model, training it, and running inference on a specific image. The right pane displays the output image with bounding boxes and confidence scores for detected objects.

```
[1] # Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # load a pretrained YOLOv8n detection model
model.train(data='coco128.yaml', epochs=3) # train the model
model('https://ultralytics.com/images/bus.jpg', save=True) # predict on an image
```

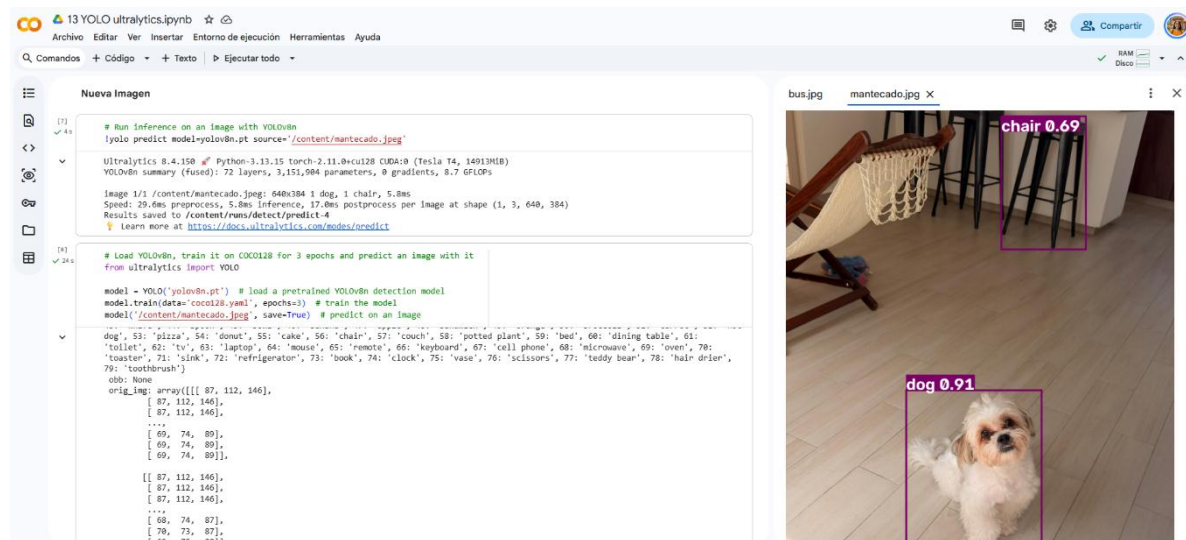
Found <https://ultralytics.com/images/bus.jpg> locally at bus.jpg
Image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 5.6ms
Speed: 1.98s preprocess, 5.6ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480)
Results saved to /content/runs/detect/predict-3
[ultralytics.engine.results.Results object with attributes:

boxes: ultralytics.engine.results.Boxes object
depth: None
keypoints: None
masks: None
names: (0: 'person', 1: 'bicycle', 2: 'car', 3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10: 'fire hydrant', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'cat', 16: 'dog', 17: 'horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 22: 'zebra', 23: 'giraffe', 24: 'backpack', 25: 'umbrella', 26: 'handbag', 27: 'tie', 28: 'suitcase', 29: 'frisbee', 30: 'skis', 31: 'snowboard', 32: 'sports ball', 33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37: 'surfboard', 38: 'tennis racket', 39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork', 43: 'knife', 44: 'spoon', 45: 'bowl', 46: 'banana', 47: 'apple', 48: 'sandwich', 49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog', 53: 'pizza', 54: 'donut', 55: 'cake', 56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'bed', 60: 'dining table', 61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'keyboard', 67: 'cell phone', 68: 'microwave', 69: 'oven', 70: 'toaster', 71: 'sink', 72: 'refrigerator', 73: 'book', 74: 'clock', 75: 'vase', 76: 'scissors', 77: 'teddy bear', 78: 'hair drier', 79: 'toothbrush')
obb: None
orig_img: array([[119, 146, 172], [121, 148, 174], [122, 152, 177], ..., [161, 171, 188], [168, 178, 187], [168, 178, 187]])

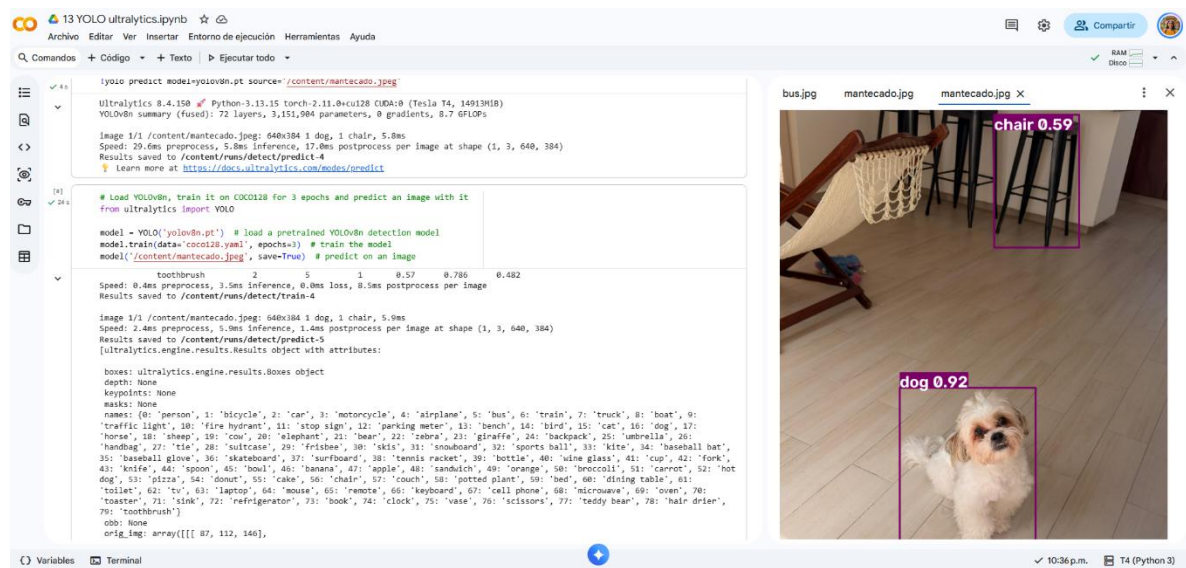
Output image (bus.jpg) showing bounding boxes and confidence scores:

- bus 0.89
- stop sign 0.41
- person 0.85
- person 0.77
- person 0.82

Mantecado – 1



Mantecado- 2



Reporte

En esta práctica utilicé YOLOv8 para realizar detección de objetos primero con las imágenes proporcionadas por Ultralytics y después con una fotografía propia. En zidane.jpg, el modelo detectó 2 personas (person) y 1 corbata (tie). En bus.jpg detectó 4 personas (person), 1 autobús (bus) y 1 señal de alto (stop sign). Para la prueba con mi propia imagen utilicé una fotografía de mi perro, Mantecado, donde YOLO detectó 1 perro (dog) y 1 silla (chair).

Algo que me llamó la atención es que en mi fotografía se pueden ver varios bancos altos al fondo, pero YOLO solamente etiquetó uno como chair. También aparece una silla tipo hamaca

bastante visible y no fue detectada. Esto puede suceder por diferentes razones. Por ejemplo, hammock no es una de las clases que maneja el modelo con COCO, mientras que otros objetos pueden no detectarse por su tamaño, posición, porque están parcialmente recortados o simplemente porque no alcanzan el umbral de confianza necesario para que YOLO los muestre como una detección.

Finalmente, probé la misma fotografía de Mantecado utilizando las dos formas que aparecen en el notebook. Con la predicción desde CLI se obtuvo 1 dog, 1 chair, y utilizando model(...) se obtuvo exactamente el mismo resultado. Por lo tanto, ambas formas de predicción coincidieron en las clases y en la cantidad de objetos detectados, aunque el tiempo de inferencia fue diferente. En general, la práctica me permitió ver que YOLO puede reconocer objetos bastante bien, pero también que el resultado depende de las clases que conoce y de qué tan claro aparece cada objeto en la imagen. Anexo la imagen de mantecado para que vea que guapo es.



Evidencias

13 YOLO ultralytics.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Comandos + Código + Texto Ejecutar todo

YOLOv8 may be used directly in the Command Line Interface (CLI) with a `yolo` command for a variety of tasks and modes and accepts additional arguments, i.e. `imgsz=640`. See a full list of available `yolo` arguments and other details in the [YOLOv8 Predict Docs](#).

[2] 14.4

```
# Run inference on an image with YOLOv8n
yolo predict model=yolov8n.pt source="https://ultralytics.com/images/zidane.jpg"
```

Downloading <https://github.com/ultralytics/assets/releases/download/v4.4.0/yolov8n.pt> to 'yolov8n.pt': 100% 6.2MB 182.1MB/s
Ultralytics 8.4.150 Python-3.11.11 torch-2.11.0 cudnn-8 (Tesla T4, 14933MiB)
YOLOv8n summary (fused): 72 layers, 3,151,904 parameters, 0 gradients, 8.7 GFLOPs

Downloading <https://ultralytics.com/images/zidane.jpg> to 'zidane.jpg': 100% 49.2KB 2.6MB/s 0.0s
Image 1/1 /content/zidane.jpg: 384x640 2 persons, 1 tie, 6.3ms
Speed: 183.5ms preprocess, 6.3ms inference, 39.2ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /content/runs/detect/predict
Learn more at <https://docs.ultralytics.com/modes/predict>

1. Detection

YOLOv8 detection models have no suffix and are the default YOLOv8 models, i.e. `yolov8n.pt` and are pretrained on COCO. See [Detection Docs](#) for full details.

[1] 37.4

```
# Load YOLOv8n, train it on COCO128 for 3 epochs and predict an image with it
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # load a pretrained YOLOv8n detection model
model.train(data='coco128.yaml', epochs=3) # train the model
model('https://ultralytics.com/images/bus.jpg', save=True) # predict on an image
```

79: 'toothbrush'
obj: None
orig_img: array([[[[119, 146, 172],
[121, 148, 174],
[122, 152, 177],
...,
[161, 171, 188],
[168, 178, 187]]]])

Recursos

No tienes una suscripción. [Más información](#)
En este momento no tienes unidades de procesamiento disponibles. Los recursos que se ofrecen sin costo no están garantizados. Compra más unidades [aquí](#).
En tu nivel de uso actual, este tiempo de ejecución puede durar hasta 3 horas 20 minutos.
[Administrar sesiones](#)

¿Quieres acceder a GPU premium? [Actualizar a Google AI Ultra](#)

(GPU del backend de Google Compute Engine en Python 3
Se muestran los recursos desde el 10:07 p.m. hasta el 10:31 p.m.)

RAM de sistema
4.2 / 12.7 GB

RAM de GPU
0.4 / 15.0 GB

Disco
47.5 / 112.6 GB

Cambiar tipo de entorno de ejecución

Variables Terminal

10:28 p.m. T4 (Python 3)